ACTIVIDAD PRÁCTICA GUIADA · UNIDAD 2

EcoPark
Control de Estadías

JavaFX · MVC · POO · Repository · DAO · JSON

Asignatura	DSY1102 – Desarrollo Orientado a Objetos
Duración sugerida	80 minutos (menos de 2 horas pedagógicas)
Modalidad	Práctica individual o guiada en laboratorio
Entorno	IntelliJ IDEA · Java 21 · Maven · JavaFX · Gson
Material inicial	Proyecto Maven con modelos y DAO implementados, vista FXML distribuida

PROPÓSITO: completar una aplicación sencilla que registre estadías de autos y motocicletas, muestre sus costos y conserve los datos en JSON.

1. Contexto del caso

El estacionamiento EcoPark necesita una aplicación de escritorio para registrar los vehículos que permanecen en sus instalaciones. El personal ingresa la patente, la cantidad de horas y el tipo de vehículo. El sistema calcula el costo según el tipo de estadía.

El estacionamiento ofrece un descuento a motocicletas con convenio. Los registros deben mostrarse en una tabla y conservarse al cerrar la aplicación.

2. Problema a resolver

Completar el proyecto base mediante la conexión entre una vista FXML ya diseñada, un Controller JavaFX, un Repository y un DAO JSON. El modelo de clases ya está desarrollado y debe reutilizarse, sin rehacerlo.

3. Reglas de negocio

Tipo	Costo	Regla especial
Auto	$2.000 × hora	Sin descuentos.
Moto	$1.000 × hora	Con convenio: descuento del 20% sobre el subtotal.

4. Requerimientos funcionales

- Registrar estadías de autos o motos mediante un ComboBox para seleccionar el tipo.
- Solicitar patente y horas. Solo se admiten horas enteras entre 1 y 24.
- Mostrar el CheckBox «Tiene convenio» como opción válida exclusivamente para motocicletas.
- Calcular y presentar automáticamente el costo de cada estadía.
- Mostrar los registros en un TableView con Tipo, Patente, Horas y Costo.
- Eliminar una estadía que haya sido seleccionada en la tabla.
- Impedir registrar dos estadías simultáneas con la misma patente.
- Conservar la información en datos/estadias.json y recuperarla al iniciar.
- Mostrar mensajes claros cuando el registro sea correcto o exista un error.

5. Diseño de interfaz

La vista ya está construida en main-view.fxml. Los estudiantes NO deben rediseñarla: deben conectar sus controles con el Controller.



6. Condiciones de validación

Campo	Validación esperada
Tipo	Debe elegirse Auto o Moto.
Patente	Obligatoria; no puede consistir solo en espacios.
Horas	Número entero entre 1 y 24.
Convenio	Solo puede aplicarse al tipo Moto.
Duplicidad	No repetir una patente ya registrada.
Eliminar	Debe existir una fila seleccionada.

7. ¿Qué viene resuelto y qué debes completar?

Componente	Estado de entrega	Trabajo del estudiante
Launcher + App	Listo	Ejecutar y comprender su función.
Model: Estadia / Auto / Moto / ConDescuento	Listo	Revisar jerarquía, interfaz y cálculos.
DAO / Gson / JSON	Listo	Entender serialización y conectarlo desde Repository.
main-view.fxml	Diseño visual listo	Agregar fx:controller, fx:id y onAction.
Repository	Esqueleto con TODO	Implementar cargar, registrar y eliminar.
MainController	Esqueleto con TODO	Inyectar controles; inicializar tabla, registrar y eliminar.
ecopark.css	Listo	Conservar el estilo disponible.

8. Secuencia de trabajo sugerida

Etapa	Tiempo	Objetivo
Explorar y ejecutar	10 min	Comprobar que la vista base abre.
Vincular FXML	12 min	Agregar fx:controller, fx:id y onAction.
Repository	13 min	Cargar, registrar y eliminar con DAO.
MainController	30 min	Validar y conectar formulario/tabla.
Pruebas	15 min	Verificar cálculos, errores y persistencia.

IMPORTANTE: la vista base no contiene fx:id, fx:controller ni onAction. Está así intencionalmente para ahorrar tiempo de diseño y dedicar la práctica a conectar las piezas.

9. Evidencias de aprendizaje

- Abstracción, herencia y polimorfismo: el cálculo cambia según el subtipo Estadia.
- Interfaz: ConDescuento es implementada por EstadiaMoto.
- FXML + Controller: los controles visuales ejecutan acciones Java.
- ObservableList + TableView: la tabla refleja las estadías en memoria.
- Repository + DAO: responsabilidades separadas para gestionar y persistir.
- Gson + JSON: los registros permanecen entre ejecuciones.

10. Casos de prueba sugeridos

Datos de entrada	Resultado esperado
Auto, ABCD12, 2 horas	$4.000
Moto, MOTO11, 3 horas, sin convenio	$3.000
Moto, MOTO22, 3 horas, con convenio	$2.400
Horas «abc», 0 o 25	Se rechaza; la aplicación no se cierra.
Patente repetida	Se rechaza con un mensaje claro.
Eliminar sin seleccionar	Se informa que debe seleccionar una estadía.
Registrar, cerrar y volver a abrir	Los registros se recuperan desde JSON.
Eliminar, cerrar y volver a abrir	El registro eliminado no reaparece.

11. Alcance y restricciones

- La aplicación contará con una sola ventana y una sola vista principal.
- Se utilizarán exclusivamente autos y motos; no se solicitan más subtipos.
- El modelo y la implementación del DAO entregados no necesitan modificarse.
- No se solicitan edición de filas, filtros, pantallas de login ni base de datos.
- Los datos se almacenarán en un archivo JSON local.
- El trabajo se realiza sobre la estructura del proyecto base proporcionado.

12. Entrega de la práctica

El estudiante deberá presentar la carpeta completa del proyecto Maven, con el FXML conectado y las operaciones principales implementadas. Para ejercicios de práctica puede realizarse una demostración directamente desde IntelliJ IDEA; si se solicita entrega digital, comprimir el proyecto sin la carpeta target/.

13. Resultado esperado

Al terminar, el estudiante podrá explicar el recorrido de una estadía desde el formulario hasta el archivo JSON y su visualización en TableView:

```
Usuario → FXML → MainController → EstadiaRepository
                                        ↓
                                  EstadiaDAO → estadias.json
                                        ↓
                                  ObservableList → TableView
```

Esta actividad es una práctica de consolidación acotada a 80 minutos. Mantiene los conceptos principales de FitPower, pero no pretende reemplazar la evaluación formal completa ni su profundidad.